

Visão computacional: o modelo, a implementação, e como transportar o método

Como a mão do usuário é convertida em três números que viram torque. O documento vai da **arquitetura do modelo** — dois modelos em cascata, não um — até a **medida invariante** que resolve o problema real, e termina no que interessa além desta prótese: **o padrão é transportável**, e a última parte mostra para onde.

Nível: técnico. Assume Python, álgebra vetorial e alguma familiaridade com câmeras. Todo número marcado como *medido* foi lido do código ou do arquivo do modelo; o que vem da documentação do MediaPipe está atribuído como tal.

MediaPipe Tasks · HandLandmarker (float16, v1) 21 landmarks · 1 mão · CPU src/thoth/perception/vision/

Parte 1	Parte 2
O modelo: por que dois modelos em cascata	A medida: por que a razão é invariante
Parte 3	Parte 4
Implementação: a cadeia completa	Orçamento de latência: onde o tempo vai
Parte 5	Parte 6
Como transportar o método para outros problemas	Falhas, calibração e depuração

PARTE 1

O modelo: por que dois modelos em cascata

O arquivo `hand_landmarker.task` parece um modelo. Não é: é um **bundle ZIP com dois modelos TFLite** e os metadados que dizem como encadeá-los. Isso é verificável — o arquivo abre com qualquer descompactador:

```
unzip -l models/mediapipe/hand_landmarker.task
 2339878  2023-04-26 03:27  hand_detector.tflite          2,23 MiB
 5478949  2023-04-26 03:27  hand_landmarks_detector.tflite 5,22 MiB
 7818827                                2 files
```

ESTÁGIO	MODELO	O QUE FAZ E POR QUE EXISTE SEPARADO
1 · Detecção	<code>hand_detector.tflite</code> BlazePalm, tipo SSD	Recebe o quadro inteiro e devolve a caixa da palma . Detecta a palma, não a mão: a palma é praticamente rígida, tem razão de aspecto quase quadrada e não tem partes articuladas, o que torna o conjunto de âncoras pequeno e o treino estável. Detectar “mão aberta ou fechada, em qualquer configuração de dedos” exigiria um detector muito maior para a mesma precisão.

ESTÁGIO	MODELO	O QUE FAZ E POR QUE EXISTE SEPARADO
2 · Landmarks	<code>hand_landmarks_detector.tflite</code>	Recebe só o recorte já centrado e escalado, e regride 21 pontos. Como a entrada é canônica — mão centrada, tamanho normalizado —, a rede não precisa aprender invariância a posição e escala; ela gasta capacidade na articulação dos dedos, que é o problema difícil.

O PADRÃO ARQUITETURAL, QUE SE REPETE EM TODA A FAMÍLIA MEDIAPIPE

Detectar o que é rígido; regredir o que é articulado. O primeiro estágio resolve *onde*, o segundo resolve *como*. A mesma divisão aparece no PoseLandmarker (detector de pessoa → 33 landmarks) e no FaceLandmarker (detector de face → 478 landmarks). Quando você for construir um pipeline de percepção próprio, essa é a primeira decomposição a tentar: ela troca um problema difícil por dois fáceis, e o segundo roda numa imagem pequena.

O truque que faz caber na CPU: o detector quase nunca roda

Em `RunningMode.VIDEO` — o modo que este projeto usa — o MediaPipe só chama o detector de palma quando *perde* a mão. Enquanto o rastreamento se sustenta, ele deriva o recorte do quadro seguinte a partir dos landmarks do quadro anterior e roda apenas o estágio 2. O modelo mais caro dos dois (5,22 MiB contra 2,23 MiB) é o que roda sempre — mas numa imagem pequena; o mais barato, sobre o quadro inteiro, roda raramente.

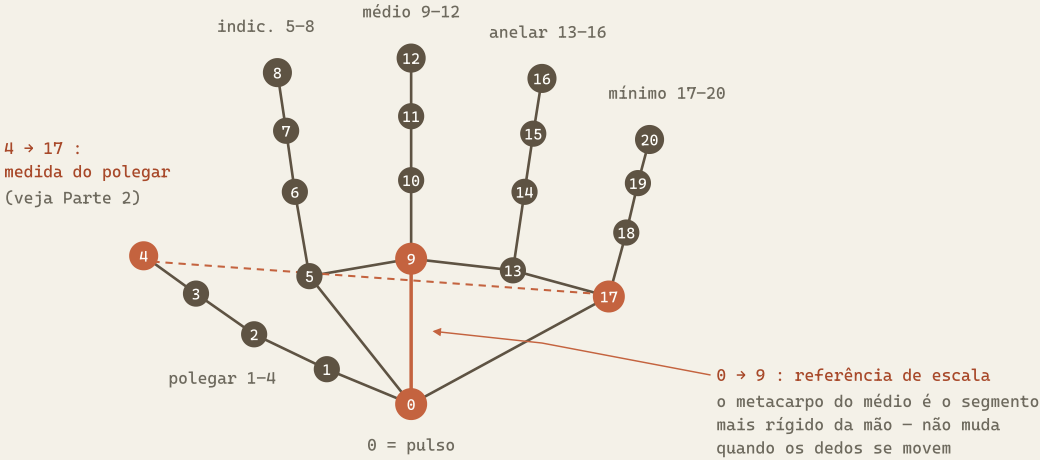
É por isso que o modo importa e é por isso que ele cobra um **timestamp monotônico crescente** a cada chamada: sem uma ordem temporal, o rastreador não consegue decidir o que é “o quadro anterior”.

O contrato de saída — e o que não confiar nele

Ler a documentação de um modelo é, principalmente, descobrir *o que ele não te dá*. Essa tabela é o que determina tudo na Parte 2.

CAMPO	UNIDADE	QUANTO CONFIAR
<code>landmarks[i].x / .y</code>	normalizado 0...1 sobre largura/altura do quadro	Alta. É a saída principal e o que este projeto usa. Note: normalizado pelo <i>quadro</i> , não pela mão — perde-se a escala absoluta.
<code>landmarks[i].z</code>	profundidade relativa, origem no pulso, escala aproximadamente igual à de <code>x</code>	Baixa. Vem de uma única câmera sem informação métrica. Serve para ordenar “mais perto / mais longe”; não serve como eixo de controle. Este projeto não usa <code>z</code> .
<code>world_landmarks</code>	metros, origem no centro geométrico da mão	Média. É uma estimativa métrica a partir de vista única, com escala inferida de um modelo de mão média. Útil para ângulos 3D aproximados; não é medição.
<code>handedness</code> + score	rótulo esquerda/direita com confiança	Média-alta com a palma visível; degrada quando a mão está de perfil.
ângulos articulares	—	Não existe. O modelo devolve posições, não ângulos. Qualquer grandeza articular é <i>sua</i> construção sobre os pontos — que é exatamente o assunto da Parte 2.

Os 21 landmarks, e os dois pares que importam aqui



Os pontos em coral são os quatro que este projeto usa como referência: **0 e 9** definem a escala da mão; **4 e 17** medem o polegar. Os outros dezessete entram apenas como pontas (8, 12, 16, 20) ou como desenho na tela.

PARTE 2

A medida: por que a razão é invariante

O modelo entrega 21 pontos em coordenadas de *imagem*. O que a prótese precisa são três escalares em $[0, 1]$. Entre uma coisa e a outra existe um problema de medição, e ele é o núcleo técnico deste documento.

O requisito, escrito com precisão

Queremos $f \in [0, 1]$ por dedo, com $f = 0$ estendido e $f = 1$ flexionado, e que f **não mude** quando variam: a distância da mão à câmera, o tamanho da mão do usuário, a posição da mão no quadro, a rotação da mão no plano da imagem e o espelhamento horizontal do vídeo. Cinco parâmetros de perturbação; nenhum deles carrega informação sobre flexão.

Por que as duas abordagens óbvias falham

TENTATIVA	POR QUE NÃO
Distância em pixels da ponta ao pulso	Sob projeção perspectiva, o tamanho aparente é $s = f \cdot L/Z$. Aproxime a mão da câmera e a distância dobra sem que nenhum dedo se mova. Falha no primeiro dos cinco parâmetros.
Ângulo articular por três pontos, via acos do produto escalar	Geometricamente correto e <i>escala-invariante</i> — mas precisa de z para ser um ângulo de verdade, e z é a saída fraca (Parte 1). Em 2D puro, o ângulo projetado colapsa quando o dedo aponta para a câmera. Além disso a derivada do acos explode perto de 0° e 180° : pequeno tremor de landmark vira grande salto de ângulo, justamente nas duas posições que mais importam.

A solução: dividir por uma referência interna

Meça a grandeza que você quer e uma referência que pertença ao mesmo objeto, e divida uma pela outra. Para o indicador:

$$r = \| p_8 - p_0 \| / \| p_9 - p_0 \|$$

ponta do indicador → pulso, dividido por pulso → base do médio

A prova de que isso resolve a escala é de uma linha. Seja s o fator de escala aparente introduzido pela projeção — o mesmo para os dois segmentos, porque eles estão à mesma profundidade aproximada. Então:

$$r = (s \cdot d_{\text{ponta}}) / (s \cdot d_{\text{ref}}) = d_{\text{ponta}} / d_{\text{ref}}$$

s se cancela — r não depende da distância Z nem do tamanho da mão

Os outros parâmetros caem por propriedades da norma euclidiana:

- **Translação:** r só usa *diferenças* de pontos, e diferença é invariante a translação. Mover a mão no quadro não muda nada.
- **Rotação no plano:** uma rotação é uma isometria — preserva normas. Logo preserva numerador, denominador e a razão.
- **Espelhamento:** reflexão também é isometria. É por isso que o `cv2.flip` aplicado ao quadro para dar “visão de espelho” **não afeta o cálculo de flexão** — uma afirmação que o código faz e que a álgebra confirma.

O LIMITE HONESTO: ROTAÇÃO FORA DO PLANO

Se a mão gira em torno de um eixo paralelo ao plano da imagem — palma inclinada para a frente ou para trás —, os dois segmentos encurtam por fatores *diferentes*, porque têm orientações diferentes no espaço. O s deixa de ser comum e a razão passa a variar sem que o dedo se mova. Isso é real e não tem conserto dentro da formulação 2D.

Mitigações, em ordem de custo: (a) operar com a palma aproximadamente de frente para a câmera — o que o espelhamento naturalmente induz, porque o usuário olha para a tela; (b) usar `world_landmarks` e calcular ângulos em 3D, aceitando a incerteza métrica; (c) duas câmeras. Este projeto escolheu (a), deliberadamente, e documenta a escolha em vez de esconder a limitação.

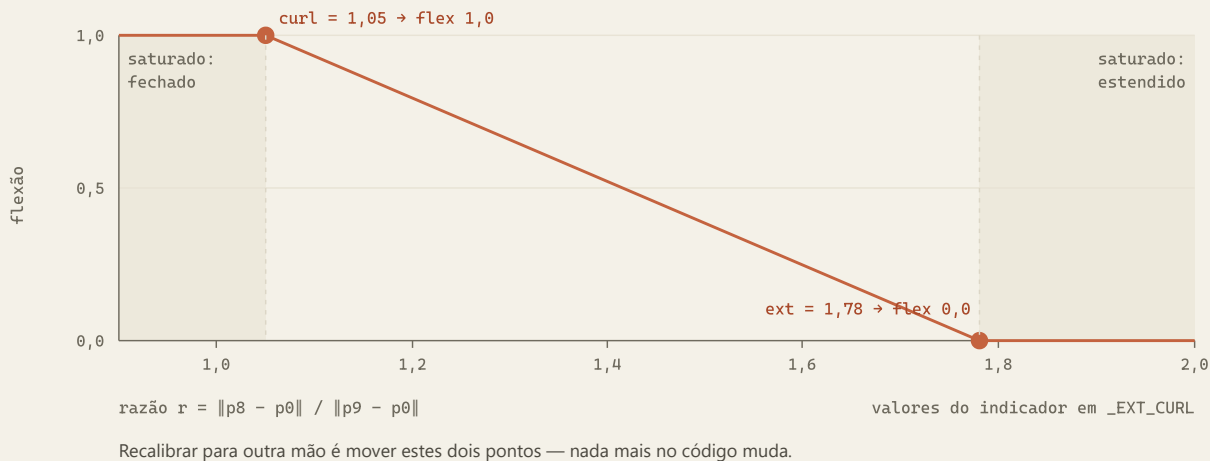
O critério de escolha do denominador

A referência tem que ser **rígida em relação ao grau de liberdade que você está medindo**. O segmento pulso(0) → base do médio(9) é o metacarpo do dedo médio: ele praticamente não se articula quando os dedos flexionam. Se em vez dele você usasse, digamos, ponta do médio(12) → pulso(0), o denominador encolheria junto com o numerador ao fechar a mão e a medida ficaria auto-referente — não monotônica, possivelmente constante. O erro é sutil e o sintoma é pior que um bug: o número *parece* funcionar e satura.

Do r para o $[0, 1]$: mapa afim com saturação

A razão é invariante mas não é uma fração — ela vive numa faixa que depende da anatomia. Dois valores-âncora, medidos empiricamente, definem o mapa:

```
def _norm(v: float, ext: float, curl: float) -> float:
    return float(np.clip((ext - v) / (ext - curl + 1e-6), 0.0, 1.0))
# ext = razão observada com o dedo TOTALMENTE ESTENDIDO -> flex 0.0
# curl = razão observada com o dedo TOTALMENTE FECHADO -> flex 1.0
# o + 1e-6 evita divisão por zero se alguém calibrar ext == curl
```



As constantes, e o que cada uma significa

DEDO	EXT (FLEX 0)	CURL (FLEX 1)	NOTA
indicador	1,78	1,05	'ext' propositalmente baixo, para o dedo conseguir chegar a 100 % aberto sem o usuário hiperextender
médio	2,10	1,05	é o dedo mais longo — razão máxima maior, pelo mesmo denominador
anelar	2,00	1,00	—
mínimo	1,80	0,95	mais curto; fecha mais "por dentro" da palma
polegar	1,25	0,55	par diferente: $\ p_4 - p_{17}\ / \ p_9 - p_0\ $ — ver abaixo

A exceção do polegar: quando a referência certa é outra

O polegar não fecha em direção ao pulso — ele **aduz atravessando a palma**. A consequência é aritmética: $\|p_4 - p_0\|$ muda pouco entre polegar aberto e polegar fechado, porque a ponta descreve um arco cujo raio em relação ao pulso é quase constante. Medir o polegar contra o pulso produz um sinal com quase nenhuma amplitude — e um sinal de baixa amplitude somado a ruído de landmark é ruído.

A troca é do *alvo*, não da matemática: mede-se a ponta do polegar (4) contra a **base do mínimo (17)**, que é o ponto para onde o polegar efetivamente se aproxima ao fechar. A amplitude passa a ser grande ($1,25 \rightarrow 0,55$, uma variação de mais de 2:1) e o sinal fica utilizável.

A REGRA GERAL EXTRAÍDA DISTO

Não meça contra a origem por hábito: meça contra **aquilo de que a coisa se aproxima**. Escolher o par de pontos é escolher onde o sinal tem amplitude — e amplitude é a única defesa gratuita contra ruído.

Agregação: três dedos, um número

Médio, anelar e mínimo compartilham **um servo** na HACKberry. A percepção respeita isso e devolve a média dos três em vez de três valores que seriam descartados:

```
other = (middle + ring + pinky) / 3.0    # 3 dedos percebidos -> 1 grau de liberdade real
```

Parece detalhe de implementação, mas é um princípio de projeto: **a dimensão da saída da percepção deve casar com o orçamento de graus de liberdade do atuador**. Estimar mais do que se pode usar custa CPU, adiciona ruído e cria a ilusão de precisão. Estimar menos joga fora capacidade do hardware.

PARTE 3

Implementação: a cadeia completa



As seis decisões de implementação que não são óbvias

01 Captura em thread, guardando só o último frame

PROBLEMA	<code>cap.read()</code> é bloqueante e a câmera produz a 30 fps independentemente de quem consome. Se o consumidor for mais lento, uma fila cresce e a latência aumenta monotonicamente — em um minuto você está espelhando o passado.
DECISÃO	Uma thread lê continuamente e sobrescreve um único slot protegido por lock; <code>read()</code> devolve uma <i>cópia</i> do último frame. Mais <code>CAP_DSHOW</code> (backend DirectShow, abertura mais estável no Windows) e <code>CAP_PROP_BUFFERSIZE = 1</code> .
PRINCÍPIO	Em percepção em tempo real, descartar frame é melhor que enfileirar frame . O frame velho não tem valor: a mão já se moveu.

02 O `import mediapipe` é adiado de propósito

PROBLEMA	O MediaPipe é uma camada C++ com logging próprio (GLOG), que lê a configuração no momento em que a biblioteca carrega . Importar no topo do módulo e configurar depois não tem efeito: o log já foi inicializado.
DECISÃO	<code>os.environ.setdefault("GLOG_minloglevel", "2")</code> no topo do arquivo, e o <code>import mediapipe</code> dentro do <code>__init__</code> do <code>HandTracker</code> .
E O GANHO	Além de silenciar avisos benignos, o processo sobe mais rápido quando a visão está desligada — o modelo só é carregado quando o espelho é ativado.

03 Timestamp monotônico: obrigatório, e o motivo do `+= 33`

PROBLEMA	Em <code>RunningMode.VIDEO</code> , <code>detect_for_video()</code> exige um timestamp em milissegundos estritamente crescente. Repetir ou retroceder um valor faz o quadro ser rejeitado.
DECISÃO	Um contador próprio, <code>self._ts_ms += 33</code> por quadro, em vez do relógio de parede.
O TRADE-OFF	Simples e imune a saltos do relógio do sistema, mas o tempo interno do rastreador <i>descola</i> do tempo real quando o pipeline não roda a 30 fps. Para este uso não importa: o rastreador só precisa de ordem, não de duração. Se algum dia entrar suavização temporal dependente de Δt , trocar por <code>time.monotonic_ns()</code> // <code>1_000_000</code> .

04 A inferência sai do event loop

PROBLEMA	<code>tracker.process()</code> é CPU-bound e bloqueante (C++). Chamado direto numa corrotina, ele congela o event loop inteiro pelo tempo da inferência — e no mesmo event loop mora o <code>_heartbeat_loop</code> que manda <code>H</code> a cada 0,3 s ao firmware.
DECISÃO	<code>annotated, flex = await asyncio.to_thread(tracker.process, frame)</code> .
CONSEQUÊNCIA SE IGNORADO	Inferências longas empilhariam atraso no heartbeat; passando de 1 s, o firmware conclui que o host morreu e abre a mão no meio do movimento . O acoplamento entre visão e segurança não é óbvio, e é por isso que está escrito aqui.

05 Banda morta de 3° e teto de ~16 Hz

PROBLEMA	Landmark tem tremor de $\pm 1\text{--}2$ px mesmo com a mão parada. Convertido em ângulo, isso vira um comando novo a cada quadro, e o servo zumba — desgaste mecânico e consumo sem movimento útil.
DECISÃO	Só envia se algum dos três ângulos mudou $\geq 3^\circ$ e se passaram 60 ms desde o último envio.
A CONTA	$115200 \text{ 8N1} = 11\,520 \text{ B/s}$. Uma linha <code>G:120,30,30\n</code> tem 13 bytes $\approx 1,13$ ms de transmissão, e cada uma provoca um <code>A:G</code> de volta. A serial suportaria centenas de comandos por segundo; o teto de 16 Hz não existe por causa dela, e sim porque acima disso o comando não tem efeito visível — ver Parte 4.

```
changed = last_angles is None or any(abs(a - b) >= _DEADBAND for a, b in zip(ang, last_angles))
if hand is not None and changed and (now - last_sent) > _SEND_PERIOD:
    await hand.set_angles(ang[0], ang[1], ang[2])      # thumb, index, other
    last_sent, last_angles = now, ang
```

06 clamp_all na saída, mesmo com o firmware já limitando

POR QUÊ Defesa em profundidade. O firmware limita porque não confia no host; o host limita porque não confia na sua própria calibração. Se alguém editar `_EXT_CURL` e produzir razões fora de faixa, a saturação do `_norm` já protege — e o `clamp_all` protege de novo, contra bug no mapa de ângulo.

DE ONDE VÊM OS LIMITES `thoth.safety.limits`, o módulo declarado *fonte única de verdade*, que espelha as constantes do firmware. A visão importa dali, não redefine.

O loop de espelhamento, e por que ele é uma máquina de estados e não um script

O `mirror_loop` roda desde o boot do serviço e não termina. Ele consulta `state.mirror_enabled` a cada volta, abre câmera e modelo *na primeira vez* que o modo é ligado e libera os dois quando é desligado. Falha ao abrir a câmera não derruba o serviço: desliga o modo, publica um evento de erro e volta a dormir.

```
while True:
    if not state.mirror_enabled:
        if cam is not None: _release()          # libera câmera e modelo
        await asyncio.sleep(0.1); continue
    if cam is None:
        try: ... abre WebcamStream + HandTracker ...
        except Exception as exc:
            state.mirror_enabled = False        # falha vira estado, não exceção que sobe
            state.push_event("espelho_erro", {"erro": str(exc)}); continue
```

Consequência prática: a câmera fica desligada — LED apagado — quando o espelho está desligado, e ligar o espelho de novo depois de um erro é um clique, não um restart do serviço.

PARTE 4

Orçamento de latência: onde o tempo realmente vai

Vale medir antes de otimizar, e o resultado aqui é contraintuitivo: **a percepção não é o gargalo**. A tabela separa o que foi medido, o que é derivado de constantes do código e o que depende da sua máquina.

ETAPA	TEMPO	ORIGEM	NOTA
período de quadro da câmera	33 ms	configurado	30 fps pedidos ao driver; o efetivo aparece em <code>state.fps</code>
conversão de cor + <code>flip</code>	< 1 ms	derivado	640×480, operação linear em memória
inferência (2 estágios)	a medir	—	o pipeline sustenta a captura a ~30 fps, o que <i>limita superiormente</i> o custo total por quadro a menos de 33 ms; instrumente com <code>time.perf_counter()</code> em volta de <code>tracker.process</code> para o número real
cálculo das 3 razões	≈ 0	derivado	doze normas de vetores 2D
portão de envio	0–60 ms	constante	<code>_SEND_PERIOD = 0.06</code>

ETAPA	TEMPO	ORIGEM	NOTA
transmissão serial	1,1 ms	derivado	13 bytes a 11 520 B/s
tick do firmware	0–20 ms	constante	<code>TICK_MS = 20</code>
curso mecânico completo	300 ms	derivado	150° de curso ÷ 10° por tick × 20 ms. É o termo dominante, por uma ordem de grandeza

A CONCLUSÃO DE PROJETO QUE SAI DESTA TABELA

O *slew-rate* do firmware — a proteção mecânica — custa 300 ms para percorrer o curso inteiro. Toda a cadeia de percepção, somada, fica na casa de 40 a 90 ms. Otimizar a inferência para ganhar 10 ms não muda nada perceptível; o movimento continua limitado pelo motor.

Isso é o que justifica o teto de 16 Hz no envio: comandos mais frequentes que o tempo de resposta mecânica são sobrescritos antes de terem efeito. E é o que justifica a banda morta: dentro de 3° o servo praticamente não se moveria de todo modo, então o comando só geraria tráfego e zumbido.

A regra transferível: meça a cadeia inteira antes de escolher o que otimizar, e inclua a mecânica na conta. O termo dominante costuma estar fora do software.

PARTE 5

Como transportar o método para outros problemas

A parte reutilizável deste projeto não é o código de espelhamento: é o procedimento. Ele aparece na literatura com outros nomes, e vale escrevê-lo como receita.

A receita: medida invariante por razão interna

1. **Nomeie a grandeza escalar e limitada** que você quer. “Quão flexionado”, “quão aberto”, “quão inclinado”. Se você não consegue escrevê-la como um número em faixa conhecida, ainda não é um problema de medição — é um problema de classificação (veja adiante).
2. **Liste os parâmetros de perturbação** — o que varia sem carregar informação sobre a grandeza. Distância à câmera, tamanho do objeto, posição no quadro, rotação, iluminação. Essa lista é o requisito de invariância, e ela precisa existir por escrito antes do código.
3. **Encontre uma referência rígida no próprio objeto:** um segmento cujo comprimento não mude quando a grandeza muda. Esse é o passo criativo, e o critério é exatamente esse — rigidez em relação ao grau de liberdade medido.
4. **Divida.** A razão cancela o fator de escala comum e herda a invariância da norma euclidiana a translação, rotação no plano e reflexão.
5. **Calibre as duas âncoras empiricamente**, com o objeto real nos dois extremos. Não derive analiticamente: anatomia, montagem e lente têm variação que o modelo geométrico não captura.
6. **Sature e amortize na fronteira física:** `clip` para a faixa, banda morta contra tremor, limite de taxa contra saturação do canal. Toda medida que vira comando

precisa dessas três.

Transferências diretas, com a mesma matemática

PROBLEMA	GRANDEZA / RAZÃO	OBSERVAÇÕES
Agachamento em [0,1] PoseLandmarker, 33 pontos	$\ \text{tornozelo} - \text{quadril} \ / \ \text{quadril} - \text{ombro} \ $	O tronco é a referência rígida em relação à flexão de joelho. Serve para contar repetições e medir amplitude sem calibrar a câmera nem saber a altura da pessoa.
Abertura de boca FaceLandmarker	$\ \text{lábio sup.} - \text{lábio inf.} \ / \ \text{canto esq.} - \text{canto dir.} \ $	A largura da boca é quase constante durante a abertura vertical — referência rígida. Vira controle contínuo de um parâmetro (volume, zoom) para acessibilidade.
Detecção de piscada	<i>eye aspect ratio</i> : altura da pálpebra / largura do olho	Exatamente o mesmo padrão, publicado como métrica EAR bem antes do MediaPipe existir. É um bom sinal de que a receita não é uma invenção local: é um idioma.
Classificação de tamanho em esteira	dimensão medida / dimensão de referência da própria peça	Dispensa calibração extrínseca da câmera e tolera variação de altura do transportador — pelo mesmo cancelamento de s .
EMG / EEG o próximo módulo do Thoth	amplitude / referência interna do próprio sujeito (contração voluntária máxima, ou linha de base em repouso)	O sinal em μV absolutos não é comparável entre sessões — impedância de pele, posição do eletrodo, gel. Normalizar por uma referência colhida do mesmo sujeito na mesma sessão é a versão elétrica da mesma ideia.

UM DETALHE QUE SALVA CONTADORES DE REPETIÇÃO

Para contar eventos a partir de uma medida contínua, não use um limiar: use **dois**. Conte a repetição quando o valor cruza 0,8 *subindo* e só rearme quando ele cair abaixo de 0,3. Com um único limiar, o tremor em torno dele gera dezenas de contagens. Isso é histerese — a mesma ideia da banda morta da Parte 3, aplicada ao domínio do evento em vez do domínio do comando.

Quando a razão *não* é a ferramenta certa

SITUAÇÃO	O QUE USAR
A perturbação dominante é rotação fora do plano	Não há razão 2D que salve. Vá para <code>world_landmarks</code> e ângulos 3D, ou estime a pose do objeto por PnP com um modelo rígido conhecido, ou use duas câmeras. Aceite a incerteza métrica da vista única.
A grandeza é categórica, não contínua “qual gesto é este?”	Um classificador sobre landmarks normalizados. Não treine sobre pixels: os 21 pontos já são a redução de dimensionalidade — ver abaixo.
Não existe referência rígida no objeto	Você precisa de escala externa: calibração com alvo de dimensão conhecida, ou profundidade métrica (câmera estéreo, ToF). Sem uma das duas, a medida absoluta não é recuperável de uma imagem.
Você precisa de valor absoluto, não relativo “quantos milímetros”	Igual ao caso anterior. A razão dá adimensional; converter em milímetro exige uma referência dimensional no mundo.

Por que o landmark é a camada mais valiosa: dados pequenos passam a ser suficientes

A saída do modelo é um vetor de $21 \times 3 = 63$ números. Comparado a treinar um CNN de ponta a ponta sobre $640 \times 480 \times 3 = 921\,600$ valores por amostra, isso é uma redução de quatro ordens de grandeza — e ela vem *de graça*, num modelo pré-treinado em milhões de mãos. A consequência prática é grande: **um classificador de gestos deixa de ser um projeto de deep learning e passa a ser um exercício de scikit-learn com algumas dezenas de exemplos por classe.**

Receita completa para um classificador de gestos novo

Antes de classificar, canonicalize — o classificador não deve gastar capacidade aprendendo invariâncias que você pode impor com três linhas de álgebra:

```
def canonical(pts):
    # pts: array (21, 2) de landmarks x,y normalizados
    p = pts - pts[0]          # 1) translação: origem no pulso -> invariante a posição
    s = np.linalg.norm(p[9]) + 1e-6
    p = p / s                  # 2) escala: pela referência rígida -> invariante a distância
    a = np.arctan2(p[9, 1], p[9, 0])  # ângulo do vetor pulso->MCP médio
    c, sn = np.cos(-a), np.sin(-a)
    R = np.array([[c, -sn], [sn, c]])
    return (p @ R.T).ravel()   # 3) rotação: canônica -> invariante a giro no plano; 42 features

# Coleta: ~50 amostras por gesto, variando distância, inclinação e iluminação.
# Treino: KNeighborsClassifier(n_neighbors=5) resolve a maioria dos casos;
#         MLPClassifier((64, 32)) se as classes tiverem sobreposição.
# Em produção: exija N quadros consecutivos com a mesma classe antes de agir
#             - a mesma histerese, agora no domínio do rótulo.
```

Se o gesto for *dinâmico* (um movimento, não uma pose), o vetor de features passa a ser a sequência — empilhe K quadros canonicalizados, ou passe as diferenças quadro a quadro. Aí sim vale um modelo temporal pequeno (GRU de uma camada, ou DTW sobre trajetórias). Continua sendo dados pequenos, porque a entrada continua sendo 42 números por quadro e não uma imagem.

O MediaPipe já traz um `GestureRecognizer`, que é literalmente um cabeçote classificador sobre estes mesmos landmarks, com gestos pré-treinados e suporte a treinar os seus. Vale checar antes de escrever o seu — o ponto aqui é entender *por que* a arquitetura permite isso.

PARTE 6

Falhas, calibração e depuração

Catálogo de falhas observadas

SINTOMA	CAUSA	O QUE FAZER
"mostre a mão para a câmera" não sai da tela	confiança abaixo de <code>min_hand_detection_confidence</code> = 0.6 : mão parcialmente fora, contraluz, ou fundo com tom de pele	melhore a iluminação frontal; aproxime a mão; se persistir, baixe para 0,4 e aceite mais falso-positivo
o polegar contrai sozinho quando a câmera liga	<code>_THUMB_EXT</code> alto demais: o polegar em repouso já cai na faixa "fechando"	foi exatamente a calibração feita neste projeto — <code>_THUMB_EXT</code> = 1.25, propositalmente baixo, para o polegar relaxado contar como aberto
a flexão satura em 0,8 e não chega a 1,0	<code>curl</code> mais baixo que a razão que a sua mão realmente alcança	feche a mão, leia a razão real e use esse valor como <code>curl</code>
o servo zumba com a mão parada	banda morta insuficiente para o tremor de landmark do seu setup	suba <code>_DEADBAND</code> para 4–5°; se o zumbido continuar sem comando novo, é energia, não visão

SINTOMA	CAUSA	O QUE FAZER
exceção de timestamp em <code>detect_for_video</code>	timestamp repetido ou decrescente	o contador tem que ser estritamente crescente por instância de tracker; ao recriar o tracker, o contador reinicia — não reaproveite valores
abre a câmera errada	índice de dispositivo diferente do esperado	<code>python scripts/check_devices.py</code> ; neste projeto a Logitech C270 é o índice 1 , não 0
a mão da prótese parece espelhada em relação à sua	é o <code>cv2.flip</code> , e é intencional	não “conserte” mexendo na flexão: o flip não afeta o cálculo (Parte 2). Se quiser desligar, é o parâmetro <code>flip</code> do <code>HandTracker</code>

Protocolo de calibração

A calibração é medição, não ajuste por tentativa. O caminho curto usa o que o código já imprime *no próprio quadro*:

1. Suba o painel (`python scripts/web.py`) e ligue o espelho. O quadro anotado mostra os landmarks e a linha `polegar NN% indicador NN% 3dedos NN%`.
2. **Estenda** a mão completamente, sem hiperextender, na distância de uso. Anote as três porcentagens. O ideal é 0 %; o que sobrar é o seu erro de `ext`.
3. **Feche** completamente. Anote. O ideal é 100 %; o que faltar é o erro de `curl`.
4. Ajuste `_EXT_CURL` e `_THUMB_EXT / _THUMB_CURL` em `hand_tracking.py`. Regra de sinal: *umentar* `ext` faz o dedo demorar mais a começar a fechar; *umentar* `curl` faz ele chegar a 100 % mais cedo.
5. Repita a 40 cm e a 80 cm da câmera. Se os números mudarem muito entre as duas distâncias, o problema não é calibração — é inclinação da palma, ou seja, o limite de rotação fora do plano da Parte 2.

O INSTRUMENTO É O DESENHO NA TELA

Os landmarks desenhados sobre o quadro e as três porcentagens impressas não são enfeite: são o instrumento de medição que torna a calibração possível em cinco minutos. É a mesma escolha do `servo_scan` no lado do firmware — diante de uma dúvida, construa o que mede, em vez de melhorar o palpite.

Limites conhecidos, declarados

- **Uma mão** (`num_hands = 1`). Duas exigiriam desambiguar qual controla o quê.
- **Sem invariância a rotação fora do plano.** O caso de uso — usuário de frente para a tela — mantém o erro pequeno, mas ele existe.
- **z não é usado.** Nenhum eixo de controle depende de profundidade estimada de vista única.
- **Sem suavização temporal.** Não há filtro de Kalman nem média móvel; o amortecimento vem da banda morta e do *slew-rate* do firmware. É mais simples e, pela Parte 4, suficiente.
- **Calibração é por mão.** As constantes valem para a mão que calibrou. Um usuário novo precisa refazer os cinco minutos do protocolo acima.

Cartão de referência

ITEM	VALOR / ONDE ESTÁ
modelo	<code>models/mediapipe/hand_landmarker.task</code> — float16, v1; bundle com 2 TFLite
baixar o modelo	<code>python scripts/download_models.py</code>
modo de execução	<code>RunningMode.VIDEO</code> , <code>num_hands = 1</code> , <code>min_hand_detection_confidence = 0.6</code> , presença e rastreo 0,5
landmarks usados	0 (pulso) · 9 (MCP médio, escala) · 4 e 17 (polegar) · 8, 12, 16, 20 (pontas)
razão do dedo	$\ p_{\text{ponta}} - p_0\ / \ p_9 - p_0\ $
razão do polegar	$\ p_4 - p_{17}\ / \ p_9 - p_0\ $
mapa flexão → ângulo	<code>lo + f · (hi - lo)</code> , com <code>lo</code> , <code>hi = 15, 165</code>
banda morta / taxa	<code>_DEADBAND = 3 °</code> · <code>_SEND_PERIOD = 0.06 s</code> (~16 Hz)
câmera	640×480 a 30 fps, <code>CAP_DSHOW</code> , <code>BUFFERSIZE = 1</code> ; índice no <code>.env</code>
arquivos	<code>perception/vision/camera.py</code> · <code>hand_tracking.py</code> · <code>mirror.py</code> · <code>safety/limits.py</code>

O modelo `hand_landmarker.task` é distribuído pelo Google (MediaPipe) e não é versionado neste repositório — o script de download busca a versão corrente. Código do projeto, guia de montagem e guia de replicação com lista de materiais estão no repositório do Projeto Thoth. Documento companheiro: *Firmware de baixo nível para a mão HACKberry*, que cobre o outro lado da serial.